

I N A T E L

Qualidade de Software

Plano de Testes

Testes automatizados de API — JSONPlaceholder

I N T E G R A N T E S

Fabio Henrique

Gabriel Baldoni

Gabriel Renato

Ian Romancini

github.com/GabrielRenat0/Projeto-Qualidade-

21 de setembro de 2026

Sumário

1. Escopo

- 1.1 Sistema testado
- 1.2 Recursos e endpoints cobertos
- 1.3 Fora do escopo

2. Objetivos

3. Estratégia

- 3.1 Abordagem
- 3.2 Particionamento de equivalência
- 3.3 Análise de valor limite
- 3.4 Testes de caracterização
- 3.5 Regras comuns a todos os casos

4. Ambiente

- 4.1 Sistema testado
- 4.2 Tecnologias e versões
- 4.3 Ambientes de execução
- 4.4 Como reproduzir

5. Descrição dos casos de teste

6. Critérios de aceite

7. Riscos e limitações

1. Escopo

1.1 Sistema testado

O JSONPlaceholder é uma API REST pública e gratuita que simula os recursos de um blog (posts, comentários, usuários, tarefas, álbuns e fotos). Ele é mantido como serviço de apoio para protótipos e treinamento, não exige autenticação e aceita tanto leitura quanto escrita. As operações de escrita são simuladas: a resposta vem como se o recurso tivesse sido criado ou alterado, mas o servidor não guarda a alteração.

Esse ponto delimita todo o trabalho. A suíte verifica o **contrato de resposta** da API, não o estado do sistema depois de cada operação.

1.2 Recursos e endpoints cobertos

Recurso	Endpoints exercitados	Casos de teste
/posts	GET /posts, GET /posts/{id}, POST /posts, PUT /posts/{id}, PATCH /posts/{id}, DELETE /posts/{id}	TC-001, TC-002, TC-003, TC-007 a TC-012, TC-014, TC-016 a TC-020
/comments	GET /posts/{id}/comments, GET /comments?postId=	TC-004, TC-015
/users	GET /users/{id}	TC-005
/todos	GET /todos?completed=	TC-006
Rota inexistente	GET /nonexistent-resource	TC-013

São 20 casos de teste, divididos em 10 com dados válidos e 10 com dados inválidos ou inoportunos. Como o TC-002 e o TC-011 são parametrizados, o relatório mostra 23 execuções.

1.3 Fora do escopo

Item	Motivo
Recursos /albums e /photos	Repetem o mesmo padrão de resposta já coberto por /posts e /comments
Autenticação e autorização	A API é aberta e não expõe nenhum mecanismo de acesso
Testes de interface	O sistema não possui interface gráfica
Testes de segurança	Não há credenciais, sessões ou dados sensíveis envolvidos
Persistência dos dados	O mock não grava alterações; essa limitação é documentada pelo TC-020 e tratada na seção 7
Desempenho dentro da suíte pytest	Coberto à parte pelos testes de carga e estresse em k6 (bônus), com relatórios em perf/results/

2. Objetivos

O objetivo geral é comprovar, por linha de comando e de forma repetível, que a API responde conforme o contrato documentado e que o comportamento diante de entradas inválidas é conhecido e registrado.

Objetivo	Como é verificado	Casos de teste
Confirmar o contrato de resposta dos recursos	Presença dos campos esperados em cada item devolvido, incluindo os objetos aninhados <code>address.geo</code> e <code>company</code>	TC-001, TC-002, TC-004, TC-005
Confirmar os códigos HTTP das operações bem-sucedidas	Asserção do status exato (200 ou 201) de cada operação	TC-001 a TC-010
Confirmar os filtros por query string	Quantidade e conteúdo dos itens devolvidos para cada filtro	TC-003, TC-004, TC-006
Confirmar as operações de escrita	Eco dos campos enviados e preservação dos campos não enviados	TC-007, TC-008, TC-009, TC-010
Verificar o comportamento com entrada inválida	Status exato e corpo devolvidos para IDs fora da faixa válida, ID não numérico, rota inexistente e filtros sem resultado	TC-011 a TC-015
Documentar as limitações do sistema testado	Asserção do comportamento observado, com o comportamento esperado de uma API real registrado em comentário no próprio teste	TC-016 a TC-020

Não é objetivo desta suíte apontar defeitos a corrigir no JSONPlaceholder: ele é um mock declarado como tal, e as divergências encontradas alimentam a seção de riscos e limitações, não um relatório de bugs.

3. Estratégia

3.1 Abordagem

A estratégia é de **caixa-preta**. Os testes atuam apenas pela interface HTTP pública, sem acesso ao código-fonte, ao banco de dados ou aos logs do servidor, e a verificação recai sobre o que a resposta expõe: status, headers e corpo. A escolha dos dados de entrada é guiada por duas técnicas de projeto de teste, aplicadas em conjunto.

3.2 Particionamento de equivalência

Os dados de entrada são agrupados em classes que a API deveria tratar da mesma forma, e cada classe é exercitada por pelo menos um caso de teste.

Partição	Exemplo de dado	Classe	Casos de teste
ID de post dentro da faixa existente (1 a 100)	1, 100	Válida	TC-002, TC-004, TC-008, TC-009, TC-010
ID de post abaixo da faixa	0	Inválida	TC-011
ID de post acima da faixa	101, 999999	Inválida	TC-011, TC-018
ID de post não numérico	abc	Inválida	TC-012
Rota existente	/posts, /users/1	Válida	TC-001, TC-005
Rota inexistente	/nonexistent-resource	Inválida	TC-013
Filtro com resultado	userId=1, completed=true	Válida	TC-003, TC-006
Filtro sem resultado	userId=9999, postId=-1	Inválida	TC-014, TC-015
Corpo de requisição completo e bem tipado	valid_post, updated_post	Válida	TC-007, TC-008
Corpo de requisição vazio, mal tipado ou malformatado	{}, title numérico, JSON quebrado	Inválida	TC-016, TC-017, TC-019

3.3 Análise de valor limite

A faixa válida de IDs de post é de 1 a 100, então os testes atacam as duas fronteiras e os valores imediatamente vizinhos, que é onde erros de comparação costumam aparecer.

Valor	Posição em relação ao limite	Resultado esperado	Caso de teste
0	Imediatamente abaixo do limite inferior	404	TC-011 (below_lower_bound)
1	Limite inferior	200	TC-002 (lower_bound)
100	Limite superior	200	TC-002 (upper_bound)
101	Imediatamente acima do limite superior	404	TC-011 (above_upper_bound)

O TC-011 ainda inclui o valor 999999 (far_above_upper_bound). Ele não é um limite: serve para confirmar que um valor bem distante da faixa recebe o mesmo tratamento que o vizinho imediato, ou seja, que a partição inválida é uniforme.

3.4 Testes de caracterização

Os TC-016 a TC-020 exercitam entradas que uma API real deveria recusar, mas que o mock aceita. Como o objetivo é documentar o sistema como ele é, esses testes afirmam o comportamento **observado** (201 onde caberia 400, 500 onde caberia 404, 200 onde caberia 404) e trazem, em comentário, o comportamento que seria esperado em produção. Essa escolha é o que liga a suíte à seção 7 do plano.

3.5 Regras comuns a todos os casos

- Os testes ficam em dois arquivos, `tests/test_valid_data.py` (TC-001 a TC-010) e `tests/test_invalid_data.py` (TC-011 a TC-020), selecionáveis pelos markers `valid` e `invalid`.
- Cada teste carrega o identificador no nome da função e na primeira linha da docstring.
- Os casos são independentes entre si: nenhum depende da execução ou do resultado de outro, e a ordem de execução não altera o resultado.
- A massa de dados fica em `data/payloads.json` e chega aos testes pela fixture `payloads`, que entrega uma cópia por teste. O `parametrize` recebe apenas rótulos, e o valor é lido da fixture dentro do teste.
- Os asserts afirmam o status exato observado, nunca uma faixa como `>= 400`.
- Antes de qualquer verificação sobre todos os itens de uma lista, a suíte confirma o tamanho esperado dela, para que uma lista vazia não faça o teste passar sem testar nada.

4. Ambiente

4.1 Sistema testado

Item	Valor
Sistema	JSONPlaceholder, API REST pública e gratuita
URL base	<code>https://jsonplaceholder.typicode.com</code>
Autenticação	Não exige
Dependências externas	Acesso à internet e disponibilidade do serviço

4.2 Tecnologias e versões

Tecnologia	Versão	Uso
Python	3.11 (execução local com 3.11.9)	Linguagem da suíte de testes
pytest	8.3.3	Execução dos testes e seleção pelos markers <code>valid</code> e <code>invalid</code>
requests	2.32.3	Cliente HTTP das chamadas à API
pytest-html	4.1.1	Relatório de execução em <code>reports/report.html</code>
k6	2.2.0	Testes de carga e estresse (bônus)
GitHub Actions	<code>actions/checkout</code> , <code>actions/setup-python</code> e <code>actions/upload-artifact</code> na versão 7	Integração contínua

As versões de `requests`, `pytest` e `pytest-html` estão fixadas no `requirements.txt`. As dependências indiretas, como `pytest-metadata` e `Jinja2`, são instaladas automaticamente pelo `pip`.

4.3 Ambientes de execução

Ambiente	Sistema operacional	Como roda
Local	Windows 11 Pro	Ambiente virtual <code>.venv</code> com as dependências do <code>requirements.txt</code>
Integração contínua	GitHub Actions, <code>ubuntu-latest</code>	A cada pull request e a cada push na <code>main</code> , com Python 3.11 instalado do zero

4.4 Como reproduzir

O `README.md` descreve a instalação e a execução: criar o ambiente virtual, instalar o `requirements.txt` e rodar `pytest`. Os testes de carga e de estresse exigem o k6 instalado e estão documentados na seção "Testes de Performance (bônus)" do README.

5. Descrição dos casos de teste

Os testes estão em `tests/test_valid_data.py` (TC-001 a TC-010) e `tests/test_invalid_data.py` (TC-011 a TC-020). Cada função de teste leva o ID no nome (ex.: `test_tc001_list_all_posts`). Os dados de entrada ficam em `data/payloads.json`, e a coluna "Dados de entrada" mostra a chave usada e o valor.

ID	Nome	Pré-condição	Dados de entrada	Ação	Resultado esperado
TC-001	Listar todos os posts	API acessível	—	<code>GET /posts</code>	200; lista com 100 posts, todos com <code>id</code> , <code>userId</code> , <code>title</code> e <code>body</code>
TC-002	Buscar post nos limites válidos de ID	API acessível	<code>valid_boundary_post_ids</code> : 1 e 100	<code>GET /posts/{id}</code>	200; <code>id</code> igual ao solicitado e campos do post presentes (2 execuções, uma por limite)
TC-003	Filtrar posts por usuário	API acessível	<code>user_posts_filter</code> : <code>userId=1</code>	<code>GET /posts?userId=1</code>	200; 10 posts, todos com <code>userId=1</code>
TC-004	Listar comentários de um post	API acessível; post 1 com comentários	<code>post_id_with_comments</code> : 1	<code>GET /posts/1/comments</code>	200; 5 comentários com <code>postId=1</code> , com <code>id</code> , <code>name</code> , <code>email</code> e <code>body</code> presentes e e-mail contendo @
TC-005	Buscar usuário com objetos aninhados	API acessível; usuário 1 existente	<code>existing_user_id</code> : 1	<code>GET /users/1</code>	200; <code>id</code> igual ao solicitado; <code>address</code> (com <code>geo.lat</code> e <code>geo.lng</code> em texto não vazio) e <code>company</code> (<code>name</code> , <code>catchPhrase</code> e <code>bs</code>) presentes
TC-006	Filtrar tarefas concluídas	API acessível	<code>completed_todos_filter</code> : <code>completed="true"</code>	<code>GET /todos?completed=true</code>	200; lista não vazia de tarefas, todas com <code>completed=True</code>

ID	Nome	Pré-condição	Dados de entrada	Ação	Resultado esperado
TC-007	Criar um novo post	API acessível	<code>valid_post</code> : objeto contendo <code>title</code> , <code>body</code> e <code>userId</code>	POST <code>/posts</code>	201; presença de um <code>id</code> numérico gerado e o eco exato dos campos <code>title</code> , <code>body</code> e <code>userId</code> enviados
TC-008	Substituir um post (PUT)	API acessível; post 1 existente	<code>updated_post</code> : objeto contendo <code>id</code> , <code>title</code> , <code>body</code> e <code>userId</code>	PUT <code>/posts/1</code>	200; <code>id</code> retornado igual a 1 e todos os demais campos atualizados exatamente como enviados no corpo da requisição
TC-009	Atualizar título do post (PATCH)	API acessível; post 1 com <code>userId=1</code>	<code>partial_patch</code> : objeto contendo apenas <code>title</code> ; <code>partial_patch_case</code> : <code>post_id</code> 1 e <code>expected_user_id</code> 1	PATCH <code>/posts/1</code>	200; campo <code>title</code> alterado de acordo com o enviado e o campo não enviado <code>userId</code> deve ser preservado intacto (mantendo o valor original 1)
TC-010	Excluir um post	API acessível; post 1 existente	<code>valid_post_id_to_delete</code> : 1	DELETE <code>/posts/1</code>	200; corpo da resposta deve ser estritamente um objeto JSON vazio <code>{}</code>
TC-011	Buscar post com ID fora da faixa válida	API acessível	<code>out_of_range_post_ids</code> : 0 (abaixo do limite inferior), 101 (acima do limite superior) e 999999 (bem acima da faixa)	GET <code>/posts/{id}</code>	404; corpo em JSON com um objeto vazio <code>{}</code> (3 execuções, uma por valor)
TC-012	Buscar post com ID não numérico	API acessível	<code>invalid_type_post_id</code> : texto <code>abc</code>	GET <code>/posts/abc</code>	404; corpo em JSON com um objeto vazio <code>{}</code>
TC-013	Acessar uma rota inexistente	API acessível	<code>nonexistent_resource</code> : <code>nonexistent-resource</code>	GET <code>/nonexistent-resource</code>	404; corpo em JSON com um objeto vazio <code>{}</code>
TC-014	Filtrar posts por usuário inexistente	API acessível; nenhum post do usuário 9999	<code>nonexistent_user_filter</code> : <code>userId=9999</code>	GET <code>/posts?userId=9999</code>	200; lista vazia <code>[]</code> , e não um erro
TC-015	Filtrar comentários por ID de post negativo	API acessível	<code>negative_comment_filter</code> : <code>postId=-1</code>	GET <code>/comments?postId=-1</code>	200; lista vazia <code>[]</code> , e não um erro
TC-016	Criar post com corpo vazio	API acessível	<code>empty_post</code> : objeto vazio <code>{}</code>	POST <code>/posts</code>	201; resposta com <code>id</code> numérico e sem <code>title</code> , <code>body</code> e <code>userId</code> . Comportamento observado do mock: não valida o payload (uma API real devolveria 400)

ID	Nome	Pré-condição	Dados de entrada	Ação	Resultado esperado
TC-017	Criar post com tipos de campo inválidos	API acessível	<code>post_with_invalid_types</code> : <code>title</code> numérico (2026) e <code>userId</code> em texto	POST <code>/posts</code>	201; <code>id</code> numérico gerado; <code>title</code> e <code>userId</code> devolvidos com os mesmos valores e tipos enviados (<code>int</code> e <code>str</code>). Comportamento observado: sem validação de schema (uma API real devolveria 400)
TC-018	Atualizar post inexistente (PUT)	API acessível; post 999999 inexistente	<code>nonexistent_post_id</code> : 999999; <code>update_nonexistent_post</code> : <code>title</code> , <code>body</code> e <code>userId</code> válidos, sem <code>id</code>	PUT <code>/posts/999999</code>	500; corpo em HTML (<code>text/html</code>) com stack trace. Comportamento observado: o mock quebra ao atualizar recurso inexistente (uma API real devolveria 404)
TC-019	Criar post com JSON malformado	API acessível; cliente envia <code>Content-Type: application/json</code>	<code>malformed_json</code> : texto <code>{title: broken json,</code> enviado cru	POST <code>/posts</code> com o corpo em <code>data=</code>	500; corpo em HTML com <code>SyntaxError</code> do parse. Sem o header JSON o mock responde 201. Comportamento observado: erro de parse não tratado (uma API real devolveria 400)
TC-020	Consultar post depois de excluí-lo	API acessível; post 1 existente	<code>post_id_to_delete</code> : 1	DELETE <code>/posts/1</code> seguido de GET <code>/posts/1</code>	DELETE 200; GET ainda 200 com <code>id</code> igual a 1. Comportamento observado: o mock não persiste a exclusão (uma API real devolveria 404)

6. Critérios de aceite

A suíte é considerada aprovada quando:

- os 20 casos de teste (TC-001 a TC-020) passam, o que corresponde a 23 execuções no `pytest`, porque o TC-002 e o TC-011 são parametrizados;
- a suíte completa roda em menos de 60 segundos;
- a execução funciona a partir de um clone limpo, seguindo apenas o README;
- o relatório HTML é gerado em `reports/`.

7. Riscos e limitações

- Sistema testado é um mock.** O JSONPlaceholder não persiste dados: no TC-020, um post excluído continua sendo devolvido com 200. O `id` gerado nas criações foi 101 nas duas execuções observadas (TC-016 e TC-017), então ele não identifica um recurso de verdade.
- Sem validação de payload.** O TC-016 (corpo vazio) e o TC-017 (tipos errados) recebem 201, e a API devolve os campos exatamente como foram enviados. Uma API real recusaria essas entradas com

400.

- **Erros 500 com detalhes internos expostos.** O TC-018 (PUT em post inexistente) e o TC-019 (JSON malformatado) devolvem 500 em HTML, com stack trace do Node e caminhos internos do servidor. Uma API real devolveria 404 e 400, sem expor esses detalhes.
- **Testes de caracterização.** Os TC-016 a TC-020 afirmam o comportamento observado do mock, e não o esperado de uma API real. Se o serviço mudar, esses testes podem falhar sem que haja um defeito no projeto. O TC-019 chega a depender do texto `SyntaxError` da resposta e do header `Content-Type: application/json` enviado pela fixture, porque sem o header o mock responde 201.
- **Dependência de internet e do serviço.** A suíte só roda com acesso à internet e com o JSONPlaceholder disponível. Uma queda ou lentidão do serviço faz os testes falharem por causa externa.
- **Sem ambiente controlado.** Não há como preparar dados, isolar o ambiente ou observar o servidor, então os testes se limitam a chamar a API pública.
- **Limites do teste de estresse.** O serviço é público e gratuito, então a carga do k6 é mantida modesta. Os resultados mostram o comportamento nesse limite, não a capacidade real do sistema. A rampa até 30 usuários virtuais não degradou o tempo de resposta (p(95) de 29,2 ms, contra 35,6 ms no teste de carga), então o que se mediu é um piso de capacidade, e não o ponto de saturação do serviço.